SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or technically

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

Smart Campus Lost-and-Found Management System

1. System Requirement Analysis

The Smart Campus Lost-and-Found Management System is intended to replace notice-board announcements and manual records used to report and recover belongings on a large campus. Students and staff can report lost items and found items with descriptions, photographs, locations, and timestamps. The system identifies possible matches, notifies potential owners, verifies ownership, records handover, and maintains recovery history and analytics.

1.1 System Objectives

- Register lost items and found items in a common system.
- Capture item descriptions, photographs, locations, and timestamps.
- Compare reports to identify potential lost-and-found matches.
- Notify possible owners and support secure item recovery.
- Verify ownership before handover and maintain recovery histories.
- Generate analytics to understand reporting and recovery trends.

1.2 Functional Requirements

Requirement	Description
FR1 – User access	Allow students/staff to sign in and submit reports; restrict administrative functions to authorized users.
FR2 – Item reporting	Create, view, update, and close lost/found reports with item category, description, photo, location, date/time, and status.
FR3 – Matching	Compare lost and found reports using category, keywords, location, time, and descriptive details; display possible matches for review.
FR4 – Notifications	Notify a potential owner when a likely match is found and provide instructions for safe contact and recovery.
FR5 – Verification and handover	Require evidence or identifying details from the claimant; record verification outcome, handover date, and responsible staff member.
FR6 – History and analytics	Maintain report and recovery history and summarize reports, matches, and successful recoveries.

1.3 Non-Functional Requirements and Quality Attributes

Attribute	Architectural requirement
Security	Authenticate users, apply role-based access, validate uploaded files, protect personal details, and use HTTPS.
Performance	Keep common report/search operations responsive; index searchable fields and paginate long result lists.
Reliability	Validate submissions and use consistent status updates so reports and handover records are not lost or duplicated.
Availability	Provide access whenever the campus service is running and perform regular backups for recovery.
Scalability	Support growth in users, reports, and photographs by separating the interface, business logic, and storage.
Maintainability	Keep matching, notification, reporting, and user-access logic in separate components with clear interfaces.

2. Selection of Suitable Software Architecture

The proposed system uses a Layered Client–Server Architecture. Users interact with a web interface, which sends requests to application services. The application layer enforces rules, processes reports, performs matching, triggers notifications, and communicates with persistent data storage. External email/SMS services may be used to deliver alerts.

This style is suitable because the system has distinct responsibilities: presentation, business processing, and data management. The separation makes the design understandable for a campus project, supports independent testing, and allows features to be improved without redesigning the whole system. A browser-based client also allows students, staff, and authorized administrators to access the same service from different devices.

2.1 Why this architecture fits

- Separation of concerns: the interface, application rules, and data storage have clear responsibilities.
- Security: authorization and validation are enforced in the application layer rather than relying only on the interface.
- Maintainability: matching rules or notification methods can be changed with limited impact on other layers.
- Scalability: the web/application service and file storage can be expanded as report volume increases.
- Practicality: a layered design is easier to implement, explain, test, and deploy than a full microservices architecture for a campus-scale system.

3. Software Architecture Diagram

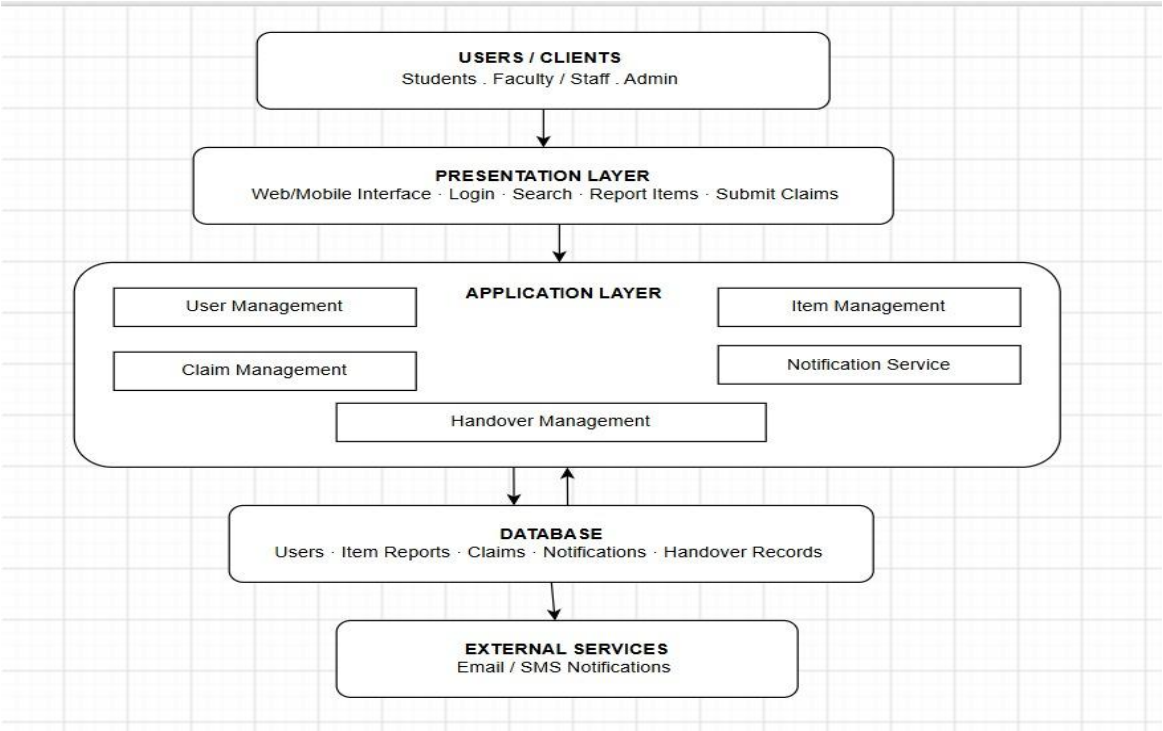


Figure 1. SYSTEM ARCHITECTURE

4. Components and Their Interactions

Component	Responsibility and interaction
User interface	Provides sign-in, lost/found report forms, search results, possible-match views, recovery status, and administrator screens. It sends validated requests to the application layer over HTTPS.
Authentication and access control	Checks user identity and role. Students/staff can create and view permitted reports; authorized staff can verify claims and manage handovers.
Item management service	Validates report fields and stores descriptions, categories, photographs or photo references, location, timestamp, and report status.
Matching service	Compares lost and found records using item category, descriptive keywords, location, and time. It produces candidate matches, not automatic proof of ownership.
Notification service	Sends alerts about possible matches through configured email/SMS or in-app notifications. It avoids exposing unnecessary personal information.
Recovery and	Guides staff through claim checks, records the verification decision,

verification service	updates the report status, and logs the handover and recovery history.
Data storage and file storage	Stores accounts, item reports, matching results, notifications, and recovery logs. Photographs can be stored as files with access-controlled references.
Reporting and analytics	Aggregates counts of lost/found reports, potential matches, recovered items, and recovery outcomes for authorized users.

4.1 Overall Workflow

1. A student or staff member signs in and submits a lost-item or found-item report with details, photo, location, and time.
2. The application validates the report and saves it with a status such as Open.
3. The matching service compares new and existing reports and returns possible matches.
4. If a possible match is found, the notification service alerts the relevant user(s) without declaring ownership automatically.
5. The claimant contacts authorized campus staff and provides item-specific evidence or identifying details.
6. Staff verify the claim, arrange a safe handover, and record the outcome; unsuccessful claims remain open for review.
7. The system updates report status and preserves the recovery history for authorized analytics.

5. Quality Attribute Evaluation

Quality attribute	How the design addresses it
Scalability	Layer separation allows the application service and photo storage to be expanded independently. Search filters, pagination, and indexed fields reduce the cost of handling larger records.
Security	Use authenticated sessions, role-based permissions, HTTPS, input validation, safe image-upload restrictions, and limited access to claimant contact details. Ownership must be verified before release.
Performance	Use indexed fields for category, location, timestamp, and status; avoid loading every report at once; run matching on relevant candidate reports.
Reliability	Validate required fields, handle errors clearly, back up records, and log important state changes. Handover updates should be saved consistently.
Maintainability	Keep authentication, item reporting, matching, notifications, and recovery logic modular. Document APIs and test each component independently.
Availability	Deploy on a dependable campus server or cloud host, monitor service health, back up data regularly, and define a recovery procedure for outages.

6. Architectural Justification, Trade-offs, and Future Scope

A layered client–server design is selected because the main work consists of user interaction, report processing, matching, notification, and secure record management. These functions can be separated while remaining part of one manageable application. The design is more straightforward to build and maintain for a campus environment than a microservices system, which would add deployment and communication overhead.

Trade-offs: a single application service can become a bottleneck if usage grows significantly, and a service outage may affect several features at once. These risks can be reduced through backups, monitoring, caching where appropriate, and scaling the application server. Matching results may include false positives, so human verification remains necessary. Notifications depend on external email/SMS availability; the system should retain notification status and allow users to check matches in the application.

Future enhancements may include improved image similarity, multilingual search, QR-coded claim receipts, integration with campus identity systems, a mobile-friendly interface, and dashboards showing recovery rates. These can be introduced through the existing service interfaces without changing the overall architecture.

Conclusion :

The proposed architecture connects item reporting, possible-match detection, notifications, ownership verification, handover records, and analytics in one structured system. Layer separation supports security, performance, maintainability, reliability, scalability, and availability. The design directly addresses the problem statement by replacing fragmented manual announcements with organized, traceable, and safer lost-and-found recovery.

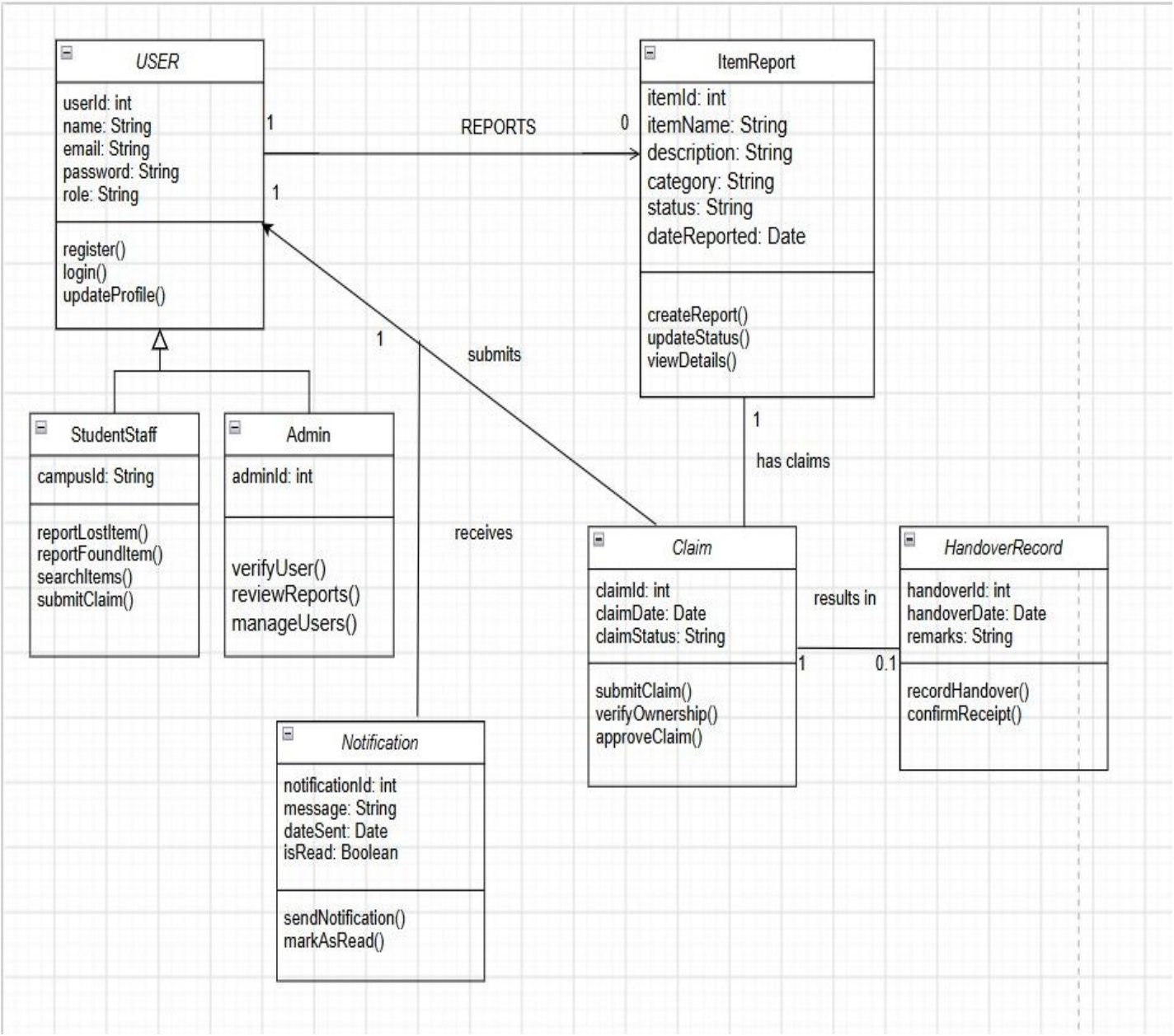


Figure 2 : CLASS DIAGRAM

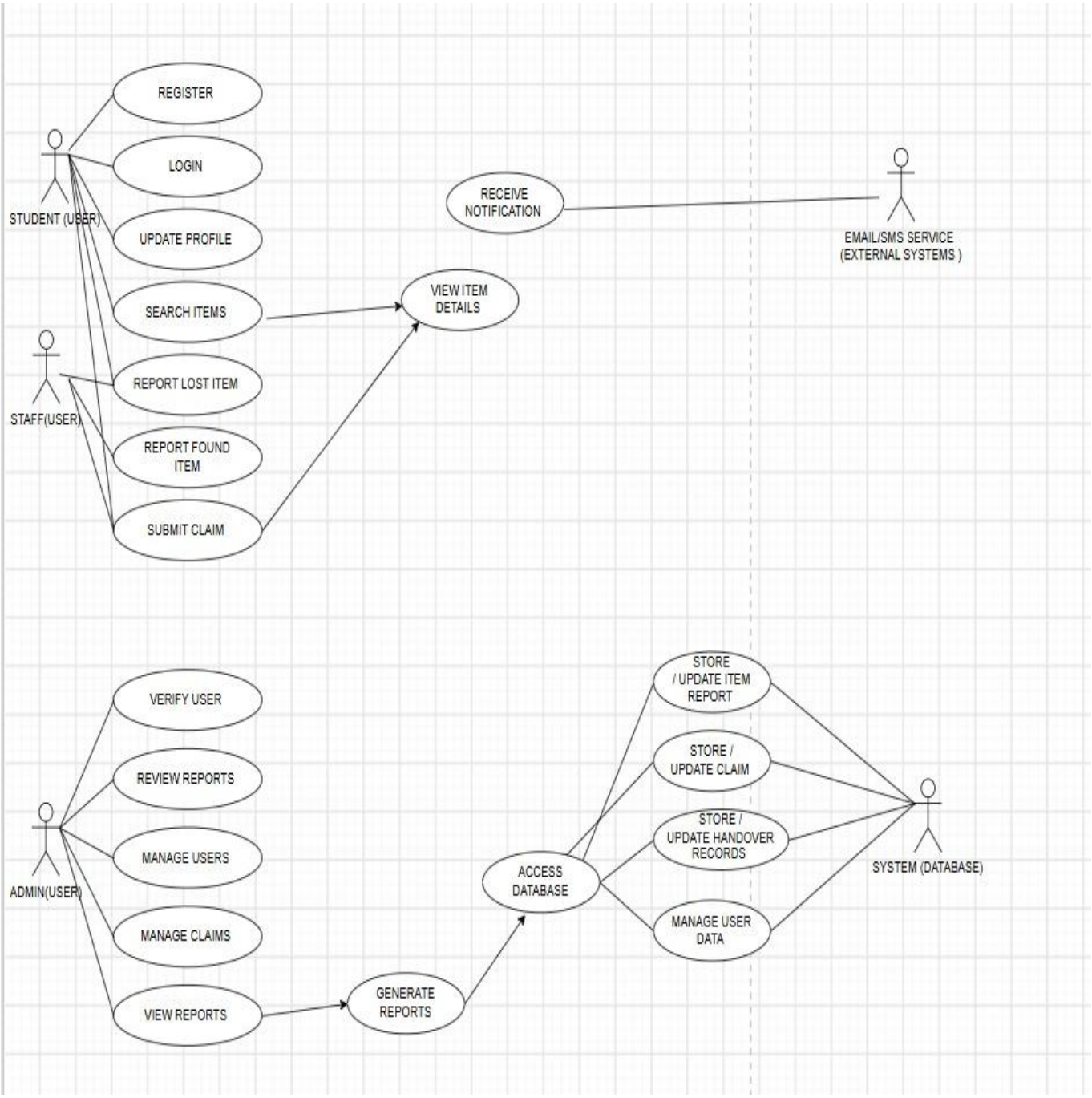


Figure 3: USE CASE DIAGRAM

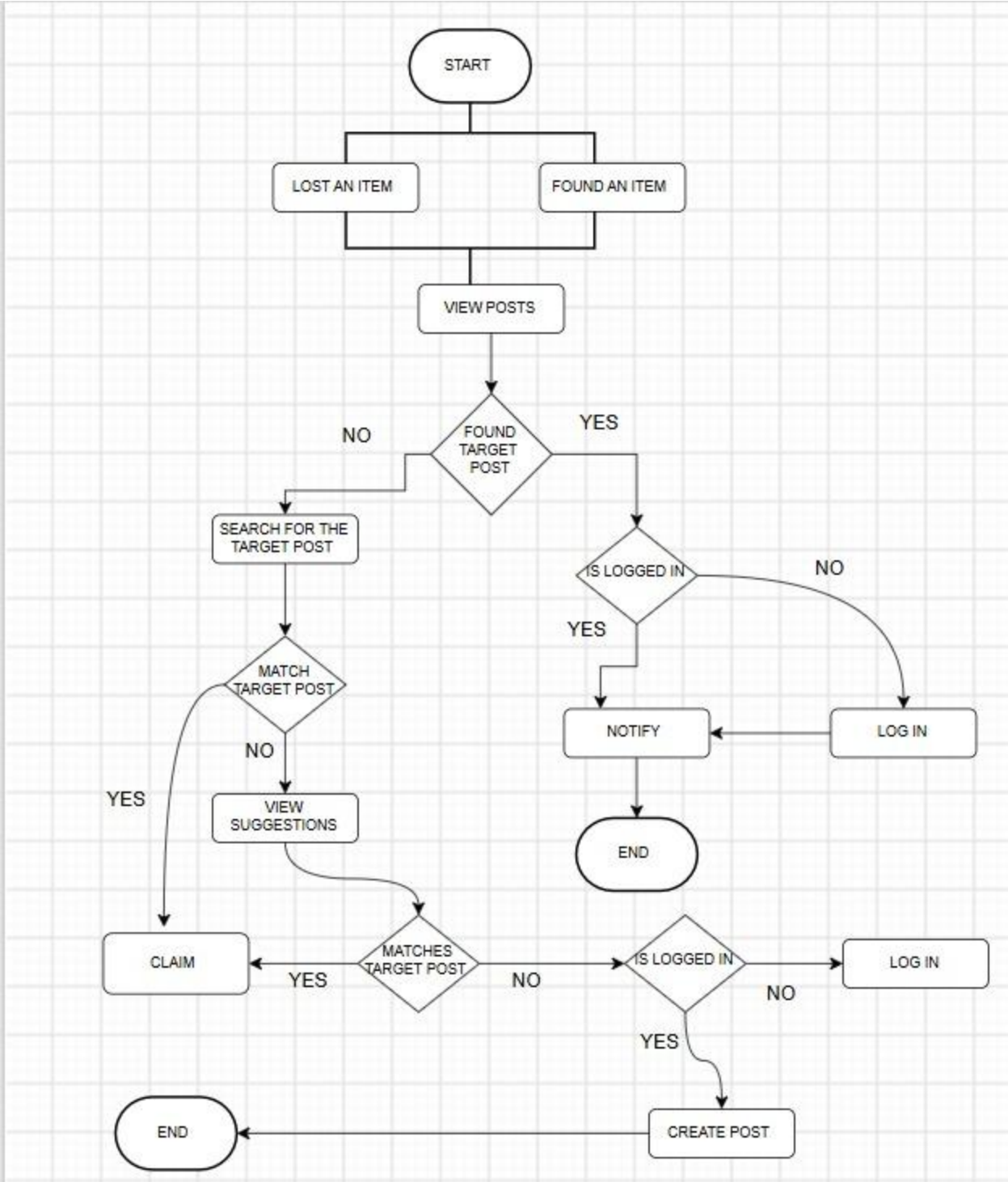


FIGURE 4 : ACTIVITY DIAGRAM